

PEN-240840116006

prac 1 Study the Data Analytics lifecycle and perform data acquisition from CSV, Excel, and JSON fles using Pandas.

step 1 import the pandas library

```
import pandas as pd

# Imports the Pandas library and assigns it the alias "pd" for easy use in data analysis
```

step 2 create sample data

```
data = {
    "ID": [101,102,103,104,105,106,107],
    "NAME":["rahul","priya","kavya","darshana","himay","prizal","dharmik"],
    "CITY":["surat","bardoli","navsari","valsad","bharuch","billimora","varachha"],
    "MARKS":[65,85,87,98,45,67,78]
}

# Creates a dictionary named "data" containing student information.
# Each key represents a column (ID, NAME, CITY, MARKS),
# and each list contains the values for that column.
```

step 3 create a data frame

```
df = pd.DataFrame(data)

df

# Creates a Pandas DataFrame from the "data" dictionary.
# Displays the DataFrame containing the student records.
```

	ID	NAME	CITY	MARKS	
0	101	rahul	surat	65	
1	102	priya	bardoli	85	
2	103	kavya	navsari	87	
3	104	darshana	valsad	98	
4	105	himay	bharuch	45	
5	106	prizal	billimora	67	
6	107	dharmik	varachha	78	

Next steps: [Generate code with df](#) [New interactive sheet](#)

step 4 save data as an csv file

```
df.to_csv("students.csv", index=False)

print("CSV file created successfully")

# Saves the DataFrame to a CSV file named "students.csv".
# Prints a confirmation message after the file is created.
```

CSV file created successfully

step 5 convert your data as an excel file

```
df.to_excel("students.xlsx", index=False)

print("Excel file created successfully")

# Saves the DataFrame to an Excel file named "students.xlsx".
# Prints a confirmation message after the file is created successfully.
```

Excel file created successfully

step 6 save data as a JSON file

```
df.to_json("students.json", orient="records", indent=4)

print("JSON file created successfully")

# Saves the DataFrame to a JSON file named "students.json".
# orient="records" stores each row as a separate JSON object.
# Prints a confirmation message after the JSON file is created successfully.
```

JSON file created successfully

step 7 read the csv file

```
df_csv = pd.read_csv("students.csv")

df_csv

# Reads the "students.csv" file into a Pandas DataFrame.
# Stores the data in the variable "df_csv".
# Displays the contents of the DataFrame.
```

	ID	NAME	CITY	MARKS	
0	101	rahul	surat	65	
1	102	priya	bardoli	85	
2	103	kavya	navsari	87	
3	104	darshana	valsad	98	
4	105	himay	bharuch	45	
5	106	prizal	billimora	67	
6	107	dharmik	varachha	78	

Next steps: [Generate code with df_csv](#) [New interactive sheet](#)

step 8 read the excel file

```
df_excel = pd.read_excel("students.xlsx")

df_excel

# Reads the "students.xlsx" Excel file into a Pandas DataFrame.
# Stores the data in the variable "df_excel".
# Displays the contents of the DataFrame.
```

	ID	NAME	CITY	MARKS	
0	101	rahul	surat	65	
1	102	priya	bardoli	85	
2	103	kavya	navsari	87	
3	104	darshana	valsad	98	
4	105	himay	bharuch	45	
5	106	prizal	billimora	67	
6	107	dharmik	varachha	78	

Next steps: [Generate code with df_excel](#) [New interactive sheet](#)

step 9 read the json file

```
df_json = pd.read_json("students.json")

df_json

# Reads the "students.json" file into a Pandas DataFrame.
# Stores the data in the variable "df_json".
# Displays the contents of the DataFrame.
```

	ID	NAME	CITY	MARKS	
0	101	rahul	surat	65	
1	102	priya	bardoli	85	
2	103	kavya	navsari	87	
3	104	darshana	valsad	98	
4	105	himay	bharuch	45	
5	106	prizal	billimora	67	
6	107	dharmik	varachha	78	

Next steps: [Generate code with df_json](#) [New interactive sheet](#)

step 10 display the first five records

```
df_csv.head()

# Displays the first 5 rows of the DataFrame.
```

	ID	NAME	CITY	MARKS	
0	101	rahul	surat	65	
1	102	priya	bardoli	85	
2	103	kavya	navsari	87	
3	104	darshana	valsad	98	
4	105	himay	bharuch	45	

step 11 display the last five records

```
df_csv.tail()

# Displays the last 5 rows of the DataFrame.
```

	ID	NAME	CITY	MARKS	
2	103	kavya	navsari	87	
3	104	darshana	valsad	98	
4	105	himay	bharuch	45	
5	106	prizal	billimora	67	
6	107	dharmik	varachha	78	

step 12 display info data frame

```
df_csv.info()

# Displays information about the DataFrame,
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0    ID      7 non-null         int64
1   NAME     7 non-null         object
2   CITY     7 non-null         object
3   MARKS    7 non-null         int64
dtypes: int64(2), object(2)
memory usage: 356.0+ bytes
```

step 13 display ststistical summary

```
df_csv.describe()

# Generates descriptive statistics for the DataFrame.
```

	ID	MARKS
count	7.000000	7.000000
mean	104.000000	75.000000
std	2.160247	17.540429
min	101.000000	45.000000
25%	102.500000	66.000000
50%	104.000000	78.000000
75%	105.500000	86.000000
max	107.000000	98.000000

step 14 display dataset shapes

```
df_csv.shape

# Returns the number of rows and columns in the DataFrame
# as a tuple in the format (rows, columns).

(7, 4)
```

step 15 display collum names

```
df_csv.columns

# Returns the names of all columns in the DataFrame as an Index object

Index(['ID', 'NAME', 'CITY', 'MARKS'], dtype='object')
```

step 16 display data types

```
df_csv.dtypes

# Displays the data type of each column in the DataFrame.
```

	0
ID	int64
NAME	object
CITY	object
MARKS	int64

dtype: object

step 17 check for missing values

```
df_csv.isnull().sum()

# Checks for missing means null values in each column.
# Returns the total number of null values for every column.
```

	0
ID	0
NAME	0
CITY	0
MARKS	0

dtype: int64

step 18 display a single column

```
df_csv["NAME"]

# Selects and displays the "NAME" column from the DataFrame.
# Returns the column.
```

```
      NAME
0    rahul
1    priya
2    kavya
3  darshana
4    himay
5    prizal
6  dharmik
```

dtype: object

step 19 display multiple column

```
df_csv[["NAME", "MARKS"]]
```

Selects and displays the "NAME" and "MARKS" columns from the DataFrame
Returns the selected columns as a new DataFrame.

```
      NAME  MARKS
0    rahul    65
1    priya    85
2    kavya    87
3  darshana    98
4    himay    45
5    prizal    67
6  dharmik    78
```



STEP 20 FILTER RECORDS

```
df_csv[df_csv["MARKS"] > 80]
```

Displays all rows where the MARKS value is greater than 80.

```
   ID  NAME  CITY  MARKS
1  102  priya  bardoli    85
2  103  kavya  navsari    87
3  104  darshana  valsad    98
```



step 21 calculate avg marks

```
df_csv["MARKS"].mean()
```

Calculates the average means mean of all values in the "MARKS" column.

```
np.float64(75.0)
```

step 22 sort data

```
df_csv.sort_values(by="MARKS", ascending=False)
```

Sorts the DataFrame by the "MARKS" column in descending order

	ID	NAME	CITY	MARKS	
3	104	darshana	valsad	98	
2	103	kavya	navsari	87	
1	102	priya	bardoli	85	
6	107	dharmik	varachha	78	
5	106	prizal	billimora	67	
0	101	rahul	surat	65	
4	105	himay	bharuch	45	

